

6. 전이 학습



• INDEX

- 전이 학습(Transfer Learning)
 - Linear Probing
 - Fine-tuning
- 모델 성능 향상
 - Data Augmentation
 - Learning Rate Scheduler

The background is a vibrant blue with a complex geometric pattern of overlapping triangles in various shades of blue. A large, white, regular hexagon is centered on the page, outlined with a thin white border. Inside this hexagon, the Korean text '전이학습' is written in a bold, black, sans-serif font.

전이학습

- ✓ 지금까지 배운 내용을 바탕으로 나만의 모델을 만들어보자!
- ✓ 나의 목표는 “고양이를 완벽하게 구분하는 모델”
- ✓ 학습을 위해 필요한 준비물
 - 강력한 GPU
 - 최신 CNN 알고리즘
 - 시간
 - 학습 데이터에 해당하는 100만장의 고품질도 이미지

➤ 자, 이제 시작해볼까요?

✓ 모델을 처음부터 학습시키는 건 현실적인 어려움이 있음

1. 방대한 양의 데이터가 필요
 - 기초부터 스스로 깨우치게 하려면 수만 장 이상의 데이터가 필요
2. 막대한 컴퓨팅 자원과 시간
 - 모든 파라미터를 업데이트하며 학습하는 시간은 고성능 GPU를 장시간 점유해야 함 + 우리의 소중한 시간
3. 느린 수렴과 초기 성능 저하
 - 랜덤 초기화 모델의 첫 에포크 정확도는 보통 10% 내외
 - local minimum에 빠질 확률이 너무 높음
4. 과적합 위험
 - 데이터가 부족하다면 학습 데이터에만 너무 맞춰진 나머지, 테스트 데이터에 대한 성능이 낮아질 수 있음

➤ 그러면 어떻게 해요???????

✓ 전이학습: 거인의 어깨에 올라타자!

- 대규모 데이터셋(예: ImageNet)으로 이미 학습된 모델의 가중치를 새로운 모델에 활용하자.
 - ImageNet: 세계에서 가장 크고 유명한 이미지 데이터베이스 (압도적 규모, 온갖 다양한 사물, 딥러닝 혁명의 시발점)
- 이미 학습된 지식을 빌려와도 문제가 없나요?
 - CNN이 학습하는 저수준 특징들(엣지, 색상, 선 등)은 매우 범용적인 공통 지식이라 활용 가능
 - 학습된 지식은 건드리지 않고, 이를 출력하는 부분만 용도에 맞게 고쳐서 활용할 수 있음



✓ 전이학습을 위한 CNN 분석

- 이미지를 분류하는 CNN은 크게 두 부분으로 나뉨

1. 특징 추출

- 합성곱과 풀링을 통해 이미지의 선, 색상, 형태, 질감 등 시각적 패턴을 찾아내는 부분
- 특징을 찾아내는 능력은 어떤 이미지에서든 공통적으로 쓰이기 때문에 **그대로 사용**

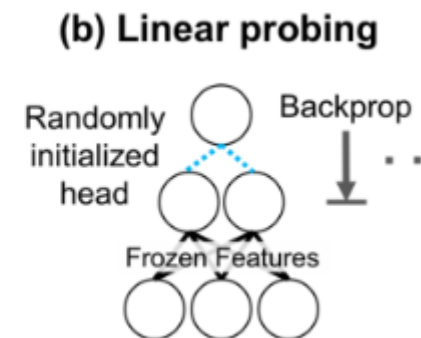
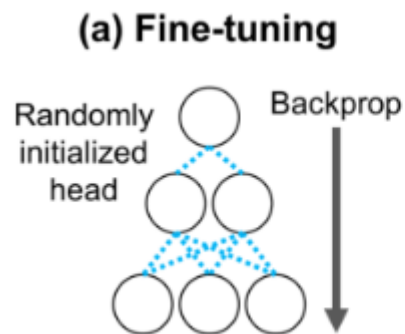
2. 분류

- 추출된 특징들을 종합하여 최종 판단을 내리는 Fully Connected Layer
- 기존 모델과 다른 목적의 분류가 필요하다면, 마지막 층만 떼어내고 우리의 목적에 맞는 **새로운 층을 달기**

- 위 특징을 이해하고, 어떻게 학습시킬 지 고민해보자!

✓ 전이학습 종류

- Linear Probing
 - 특징 추출부의 가중치를 고정(Freeze)하고, 새로 만든 분류층만 학습
 - 학습할 양이 매우 적어 압도적으로 빠름
 - 학습 데이터가 아주 적어서 빌려온 모델을 망가뜨릴 것 같을 때!
- Fine-tuning
 - 고정했던 특징 추출부까지 포함하여 모델 전체를 다시 학습
 - 내 데이터에 완전히 최적화되므로 최고의 성능을 발휘
 - 기존 지식을 잊어버리면 안되므로, 조심스럽게 학습을 진행해야 함



Linear Probing

✓ Linear Probing

- 사전 학습된 모델의 “특징 추출부” 가중치를 고정(Freeze)하고, 마지막에 새로 추가한 분류층만 학습시키는 방법
- 동작 방식
 - **가중치 동결**(Weight Freezing)
 - 모델의 합성곱(Convolution) 층들의 파라미터가 업데이트되지 않도록 설정
 - 이미 ImageNet 같은 데이터로 최적화된 “특징 추출 능력”을 훼손하지 않기 위해서
 - code: `param.requires_grad = False`
 - **분류층 교체**(Head Replacement)
 - 기존 출력 노드를 제거하고, 우리의 목적에 맞는 출력 노드를 가진 새로운 층을 붙임
 - 이 층은 초기에 랜덤한 값으로 초기화되어 있으므로, 새로운 데이터를 학습시켜야 함

✓ Linear Probing을 하는 이유

- 압도적인 학습 속도
 - 전체 파라미터 중 약 1% 미만(분류층)만 학습하면 되기 때문에 GPU 자원을 적게 점유
- 적은 데이터로도 가능
 - 데이터가 부족할 때 모델 전체를 학습하면 과적합이 될 수 있지만, 분류층만 학습하면 되기 때문에 훨씬 안정적
- 성능의 베이스라인
 - “사전 학습 모델이 얼마나 괜찮은가?” 를 확인하는 객관적인 지표가 됨
 - 예를 들어, ResNet과 ViT의 성능을 비교하고 싶다면 둘 다 똑같이 올려놓고 Linear Probing 하면 성능을 비교할 수 있음

✓ Linear Probing Code

```
# 1. 마지막 분류층(Fully Connected Layer) 교체
# ResNet-18의 마지막 층을 수행해주세요.
# 기존 ResNet-18의 마지막 층(model.fc)은 ImageNet의 1000개 클래스를 분류합니다.
# 우리는 CIFAR-10의 10개 클래스를 분류하도록 새로운 `nn.Linear` 레이어로 교체해야 합니다.
model.fc = nn.Linear(model.fc.in_features, 10)

# 2. 선형 프로빙을 위해 ResNet-18을 1에서 선언한 마지막 층을 제외하고 업데이트가 되지 않도록 가중치를 동결하세요.
# `param.requires_grad = False`로 설정하면 해당 파라미터는 학습 중에 업데이트되지 않습니다.
for name, param in model.named_parameters():
    if "fc" not in name: # fc 층이 아닌 경우
        param.requires_grad = False
```

Fine-tuning

✓ Fine-tuning

- 동결 했던 “특징 추출부”도 포함해서 모델의 전체 파라미터를 우리 데이터셋에 맞춰 다시 학습시키는 방법
- 전체 레이어의 동결을 해제해야 함
- 가져온 사전 학습 모델은 매우 정교하기 때문에 학습률을 매우 낮춰서 학습을 진행해야 함
 - 학습률이 높으면 기존 가중치를 순식간에 파괴하여 성능이 급락할 수 있음

```
# 1. 전체 레이어의 동결을 해제해주세요
for param in model.parameters():
    param.requires_grad = True

# 2. 새로운 `optimizer`를 선언해주세요. 학습률은 낮은 0.0005로 설정해주세요.
optimizer = optim.SGD(model.parameters(), lr=0.0005)
```

- 기존 성능이 이미 좋지만 Fine-tuning을 사용하는 이유
 - 우리 데이터만의 특수한 질감이나 형태에 더 예민하게 반응하도록 하기 위해서

✓ Linear Probing vs Fine-tuning

구분	Linear Probing	Fine-tuning
정의	특징 추출 부분(Backbone)을 동결하고 분류층(Head)만 학습	전체 또는 일부 모델을 새로운 데이터로 추가 학습
학습 파라미터	분류층만 학습 (전체의 약 1% 미만)	모델 전체 학습 (100%)
학습 속도	매우 빠름 (분 단위)	상대적으로 느림 (시간 단위)
학습률 (LR)	상대적으로 큼 (0.001 ~ 0.01)	상대적으로 작음 (0.0001 ~ 0.001)
성능	기본 성능 확보	최고 성능 달성 가능
과적합 위험	낮음	높음 (특히 데이터가 적을 때)
적용 시나리오	데이터가 적거나 빠른 프로토타이핑이 필요할 때	충분한 데이터와 컴퓨팅 자원이 있을 때



모델 성능 향상

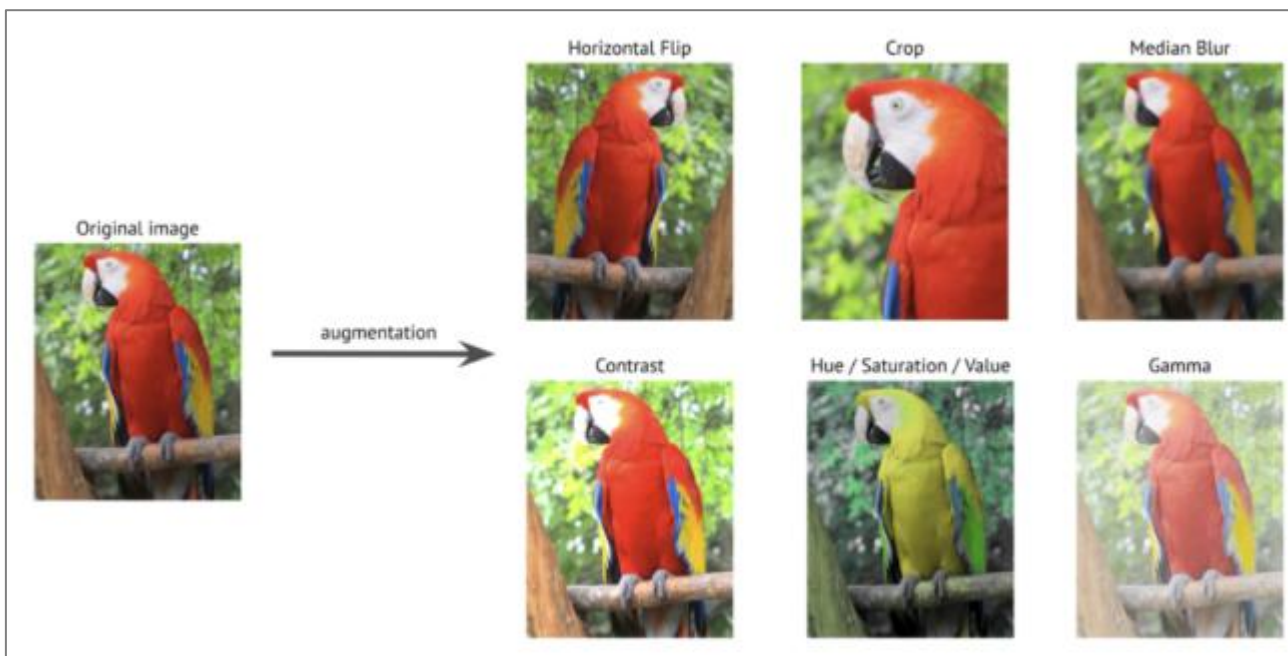
✓ 앞서 배운 전이학습을 토대로 다시 학습을 시작해보자!?

- 현실은 전체 모델을 학습시키기 전 두 가지 큰 문제에 직면하게 됨
 - 과적합(Overfitting): 모델이 데이터를 암기해서 테스트에서는 성능을 발휘하지 못하는 것
 - 최적화(Optimization): 너무 빠르거나 느리게 학습하여 최적의 성능을 놓치는 것
- 성능 향상을 위한 두 가지 기술
 - 데이터 증강(Data Augmentation)
 - 학습률 스케줄러(LR Scheduler)

데이터 증강

✓ 데이터 증강(Data Augmentation)

- 학습 데이터에 무작위적인 변환(회전, 자르기, 뒤집기 등)을 가하여 데이터의 다양성을 인위적으로 높이는 기법
- 모델은 매번 조금씩 다른 이미지를 보게 되므로, 사물의 본질적인 특징을 학습하게 됨



✓ 데이터 증강 특징

- [장점] 과적합(Overfitting) 방지
 - 데이터에 노이즈를 주어 암기를 방해하고 이해를 유도함
- [장점] 일반화 성능 향상
 - 실제 세상의 사물은 회전/자르기/뒤집기 등 다양한 노이즈가 있는데, 증강은 다양한 상황에서 모델이 유연하게 대처할 수 있게 함
- [장점] 부족한 데이터 극복
 - 부족한 데이터 수를 증강을 통해 무한에 가까운 조합으로 변경시켜 학습 효과를 극대화할 수 있음
- [단점] 데이터의 본질을 훼손
 - 이미지의 모양을 바꿀 수 있지만, 사물의 정체성 또한 바뀔 수 있음 (ex: 6을 뒤집으면 9)
- [단점] 학습 난이도가 늘어나고, 학습 시간이 늘어남
 - 학습이 매우 느리게 진행되어 손실 값이 잘 떨어지지 않을 수 있음

✓ 데이터 증강 코드

```
train_transform_aug = transforms.Compose([
    # 두 가지 증강기법을 추가해봅시다. 기존의 `train_transform`의 구성에 추가해주세요.
    # 32x32 이미지 주변에 4픽셀의 패딩(padding)을 추가한 뒤, 무작위로 32x32 영역을 잘라냄
    transforms.RandomCrop(32, padding=4),
    # 50% 확률로 이미지를 좌우로 뒤집음
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=mean, std=std)
])
```

학습룰 스케줄러

✓ 학습률 스케줄러(Learning Rate Scheduler, LR Scheduler)

- 특정 시간이나 Epoch에 따라 학습률(Learning Rate, 보폭)을 동적으로 조정하는 기법
- 처음에는 크게 움직이다가, 정답 근처에 가서는 조심스럽게 작게 움직이는 것이 핵심
- 학습 초반에는 “학습률을 크게” 설정하여 넓은 영역을 빠르게 학습
- 학습 후반에는 “학습률을 작게” 설정하여 천천히 학습(빠르면 정답을 지나치기 때문)
- 주요 스케줄러 종류
 - StepLR: 일정한 에포크마다 학습률을 고정된 비율로 감소시킴 (구현이 쉽고, 직관적)
 - CosineAnnealingLR
 - ReduceLROnPlateau

✓ 학습률 스케줄러(LR Scheduler) 특징

- [장점] 세밀한 손실 감소
 - 학습률이 줄어드는 시점마다 손실값이 확 떨어지는 걸 볼 수 있음
- [장점] 성능 극대화와 안정적인 학습
- [단점] 또 하나의 파라미터 지옥
 - 스케줄러를 도입하는 순간, 우리가 관리해야 할 파라미터가 하나 더 추가됨
 - 이 수치가 조금만 어긋나도 모델이 충분히 학습되지 않거나, 너무 늦게 수렴되어 시간만 낭비하게 됨
- [단점] 성급한 학습 종료
 - 모델이 더 학습해야 되는 상태이지만 스케줄러가 학습률을 줄여서 정답 근처에도 가지 못하는 경우가 생김
- [단점] 범용성 부족
 - “5에포크마다 0.1배씩 줄이는 게 정석”과 같은 일반화 케이스가 없음
 - 즉, 본인이 실험을 통해서 적절한 파라미터 값을 찾아야 함

✓ Optimizer vs LR Scheduler (1/2)

- Optimizer(Gradient Descent, Adam, ...)
 - 매 순간(Batch)마다 들어오는 데이터의 기울기를 보고, 개별 파라미터들을 조정
- LR Scheduler(StepLR, ...)
 - 특정 시간(Epoch)가 지나거나 성능이 정체되면, 기본 학습률 자체를 조정

구분	옵티마이저 (Optimizer)	학습률 스케줄러 (LR Scheduler)
핵심 질문	"어느 방향으로, 어떻게 움직일까?"	"지금 속도를 줄여야 할 때인가?"
관리 대상	모델의 가중치(Weights) 하나하나	옵티마이저가 가진 기본 학습률(LR)
실행 단위	매 배치(Batch)마다 (실시간)	매 에포크(Epoch)마다 (주기적)
실습 코드	<code>optimizer.step()</code>	<code>scheduler.step()</code>

✓ Optimizer vs LR Scheduler (2/2)

- 예를 들어, **기준 학습률**이 0.01로 설정되어있다고 가정
- Optimizer(Adam)의 작업
 - 가중치별로 보폭을 다르게 가져감
 - 가중치 W_1 (기울기가 가파름): $0.01(\text{기준}) * 0.5(\text{Adam 계수})$
 - 가중치 W_2 (기울기가 완만함): $0.01(\text{기준}) * 2.0(\text{Adam 계수})$
- LR Scheduler(StepLR)의 작업
 - 초기 세팅: `step_size=5, gamma=0.1`
 - **기준 학습률**이 0.01에서 0.001로 바뀜